Relatório de Análise

Características de Repositórios Populares do GitHub

Análise empírica dos 1.000 repositórios com maior número de estrelas

Disciplina	Experimentação de Software
Instituição	PUCMINAS
Período	01/2026
Autores	Augusto Fuscaldi Cerezo, Filipe Faria Melo

Índice

1. Introdução
- **1.1** Contextualização
- **1.2** Problema Foco do Experimento
- **1.3** Questões de Pesquisa
- **1.4** Hipóteses
- **1.5** Objetivos

2. Metodologia
- **2.1** Passo a Passo do Experimento
- **2.2** Decisões Metodológicas
- **2.3** Materiais Utilizados
- **2.4** Métodos Utilizados
- **2.5** Métricas e suas Unidades

3. Visualização dos Resultados
- **3.1** Dados Coletados
- **3.2** Estatísticas Descritivas Gerais
- **3.3** Análise por Linguagem (RQ07)

4. Discussão dos Resultados
- **4.1** RQ01 – Maturidade dos Sistemas Populares
- **4.2** RQ02 – Contribuição Externa
- **4.3** RQ03 – Frequência de Releases
- **4.4** RQ04 – Frequência de Atualização
- **4.5** RQ05 – Linguagens de Programação
- **4.6** RQ06 – Gestão de Issues
- **4.7** RQ07 – Análise Comparativa por Linguagem

5. Conclusão
- **5.1** Tomada de Decisão
- **5.2** Sugestões Futuras

1. Introdução

1.1 Contextualização

O GitHub é a maior plataforma de hospedagem de código-fonte do mundo, abrigando milhões de repositórios open-source. Repositórios com alto número de estrelas tendem a ser mais populares e amplamente utilizados pela comunidade. Entender as características desses sistemas populares pode fornecer insights valiosos sobre boas práticas de desenvolvimento, manutenção e gestão de projetos open-source.

1.2 Problema Foco do Experimento

Há um interesse sobre quais características técnicas e de manutenção são comuns entre os sistemas open-source mais populares. Questões sobre maturidade, frequência de atualização, linguagens utilizadas e gestão de contribuições externas permanecem sem respostas claras baseadas em dados empíricos.

1.3 Questões de Pesquisa

Código	Questão de Pesquisa
RQ01	Sistemas populares são maduros/antigos?
RQ02	Sistemas populares recebem muita contribuição externa?
RQ03	Sistemas populares lançam releases com frequência?
RQ04	Sistemas populares são atualizados com frequência?
RQ05	Sistemas populares são escritos nas linguagens mais populares?
RQ06	Sistemas populares possuem alto percentual de issues fechadas?
RQ07	Existe correlação entre linguagem de programação e métricas de contribuição/atualização?

1.4 Hipóteses

	Hipótese
H1	Sistemas populares tendem a ser maduros, com idade superior a 5 anos.
H2	Sistemas populares recebem alta contribuição externa (mediana > 200 PRs aceitas).
H3	Sistemas populares possuem releases frequentes (mediana > 20 releases).
H4	Sistemas populares são atualizados frequentemente (mediana < 15 dias desde último push).
H5	Sistemas populares utilizam linguagens mainstream como JavaScript, Python, Java.
H6	Sistemas populares possuem alto percentual de issues fechadas (mediana > 70%).
H7	Linguagens diferentes apresentam padrões distintos de contribuição e atualização.

1.5 Objetivos

Objetivo Principal: Caracterizar sistemas open-source populares através de análise empírica dos 1000 repositórios com maior número de estrelas no GitHub.

Objetivos Específicos:

- Coletar dados quantitativos sobre idade, contribuições, releases, atualizações, linguagens e issues
- Calcular estatísticas descritivas (média, mediana, mínimo, máximo) para cada métrica
- Identificar padrões e correlações entre linguagens de programação e métricas de desenvolvimento
- Validar ou refutar as hipóteses formuladas através de análise de dados
- Fornecer insights para decisões de gestão de projetos open-source

2. Metodologia

2.1 Passo a Passo do Experimento

Etapa 1 – Planejamento

- Definição das questões de pesquisa (01–07)
- Identificação das métricas necessárias
- Formulação de hipóteses

Etapa 2 – Coleta de Dados

- Autenticação na API GraphQL v4 do GitHub usando token de acesso
- Query dos 1000 repositórios com maior número de estrelas
- Paginação de 10 repositórios por requisição (100 requisições totais)
- Delay de 2 segundos entre requisições para respeitar rate limiting
- Sistema de retry com backoff exponencial (3 tentativas)
- Armazenamento dos dados em estruturas Python (lista de dicionários)

Etapa 3 – Processamento

- Cálculo de métricas derivadas (idade em dias, dias desde último push, percentual de issues)
- Agregação de estatísticas descritivas por métrica
- Contagem de repositórios por linguagem
- Análise estratificada por linguagem (top 5)

Etapa 4 – Exportação

- Geração de arquivo CSV com dados brutos e estatísticas
- Organização dos resultados para análise

Etapa 5 – Análise

- Análise de valores medianos para cada RQ
- Comparação com hipóteses formuladas
- Identificação de padrões e outliers
- Documentação de insights

2.2 Decisões Metodológicas

Decisão	Justificativa
Critério de Popularidade	Número de estrelas foi escolhido como proxy de popularidade por ser uma métrica amplamente aceita e de fácil coleta.

Tamanho da Amostra	1000 repositórios foram selecionados para balancear representatividade e viabilidade de coleta dentro dos limites da API.
Métrica de Atualização	Optou-se por pushedAt ao invés de updatedAt, pois este último é modificado por qualquer atividade (stars, forks), enquanto pushedAt reflete commits de código efetivos.
Pull Requests	Consideradas apenas PRs com estado MERGED para medir contribuições efetivamente aceitas.
Issues	Calculado percentual de fechamento para normalizar entre repositórios de tamanhos diferentes.
Estatística Central	Mediana foi escolhida como medida principal por ser robusta a outliers, comum em distribuições de dados de software.

2.3 Materiais Utilizados

Item	Detalhes
API	GitHub GraphQL API v4 — Endpoint: https://api.github.com/graphql Autenticação: Token de acesso pessoal com permissão public_repo
Linguagem	Python 3.12
Bibliotecas	requests, csv, datetime, tkinter, threading
Infraestrutura	Sistema Operacional: Windows, conexão de internet estável, token válido do GitHub

2.4 Métodos Utilizados

- Coleta de Dados: Query GraphQL parametrizada com cursor de paginação.
- Tratamento de Erros: Retry com backoff exponencial (1s, 2s, 4s).
- Processamento Temporal: Cálculo de diferença em dias entre data atual e timestamps ISO 8601.
- Agregação Estatística: Cálculo de média, mediana (percentil 50), mínimo e máximo.
- Categorização: Agrupamento de repositórios por linguagem primária.

2.5 Métricas e suas Unidades

Métrica	Unidade	Descrição
Idade do Repositório	Dias	Diferença entre data atual e createdAt
Pull Requests Aceitas	Contagem	PRs com estado MERGED
Total de Releases	Contagem	Número de releases publicadas
Dias desde Último Push	Dias	Diferença entre data atual e pushedAt
Linguagem Primária	Categoria	Linguagem com maior % de código
Percentual Issues Fechadas	% (0–100)	(issues fechadas / total) × 100

Número de Estrelas	Contagem	Total de stars do repositório

3. Visualização dos Resultados

Total de repositórios analisados: **1.000**

3.2 Estatísticas Descritivas Gerais

RQ01 – Idade dos Repositórios

Estatística	Valor (dias)	Valor (anos)
Média	2.290	~6,27
Mediana	3.062	~8,39
Mínimo	46	~0,126
Máximo	6.537	~17,9

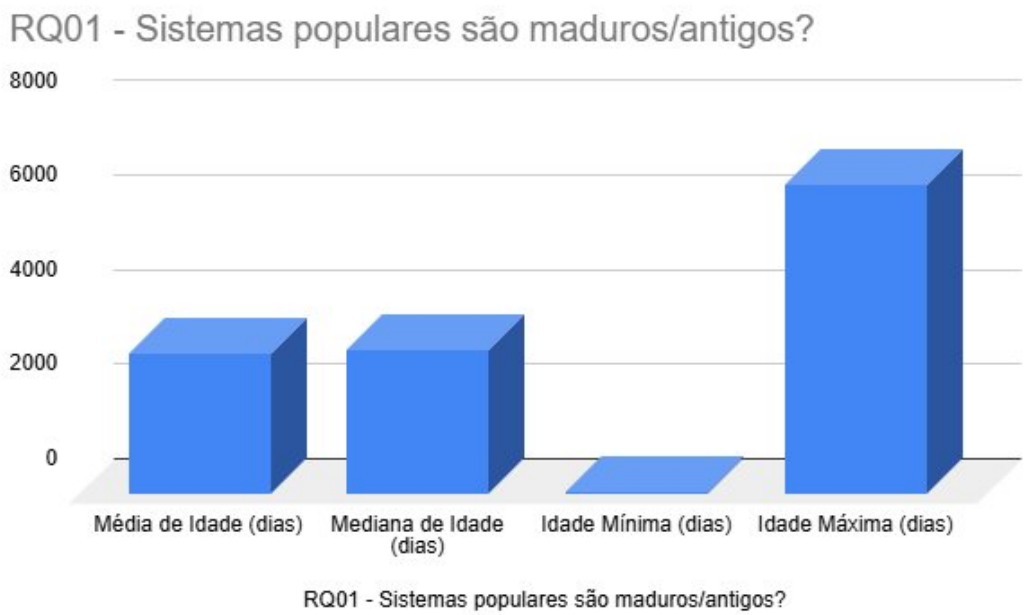

Figura 1 – RQ01: Distribuição de idades dos repositórios

RQ02 – Pull Requests Aceitas

Estatística	Valor
Média	3.954
Mediana	741
Mínimo	0
Máximo	94.634

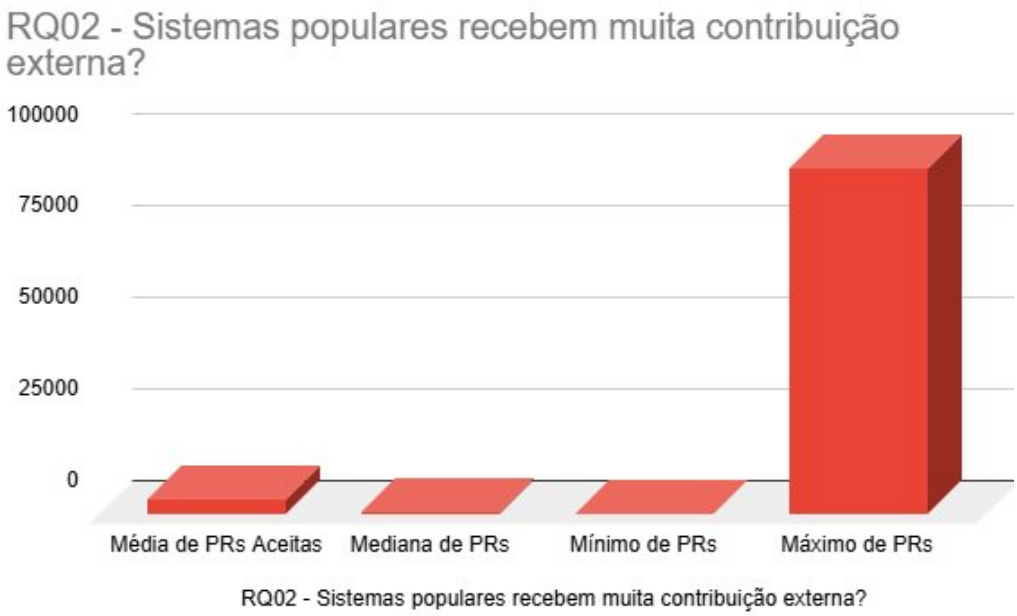

Figura 2 – RQ02: Pull Requests aceitas

RQ03 – Releases

Estatística	Valor
Média	120
Mediana	41
Mínimo	0
Máximo	1.000

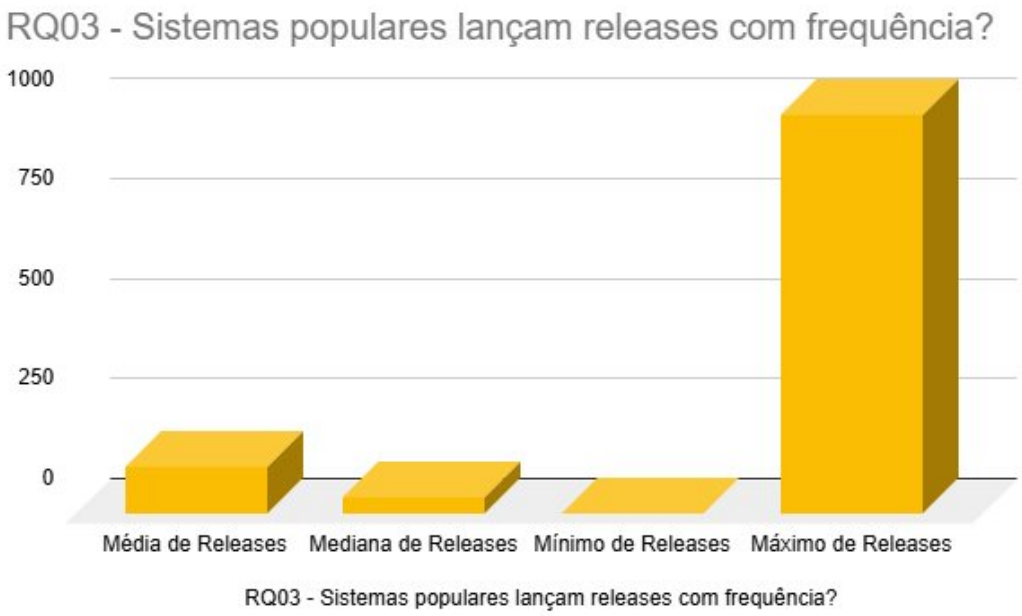

Figura 3 – RQ03: Total de releases por repositório

RQ04 – Frequência de Atualização

Estatística	Valor (dias)
Média	111
Mediana	1
Mínimo	0
Máximo	2.284

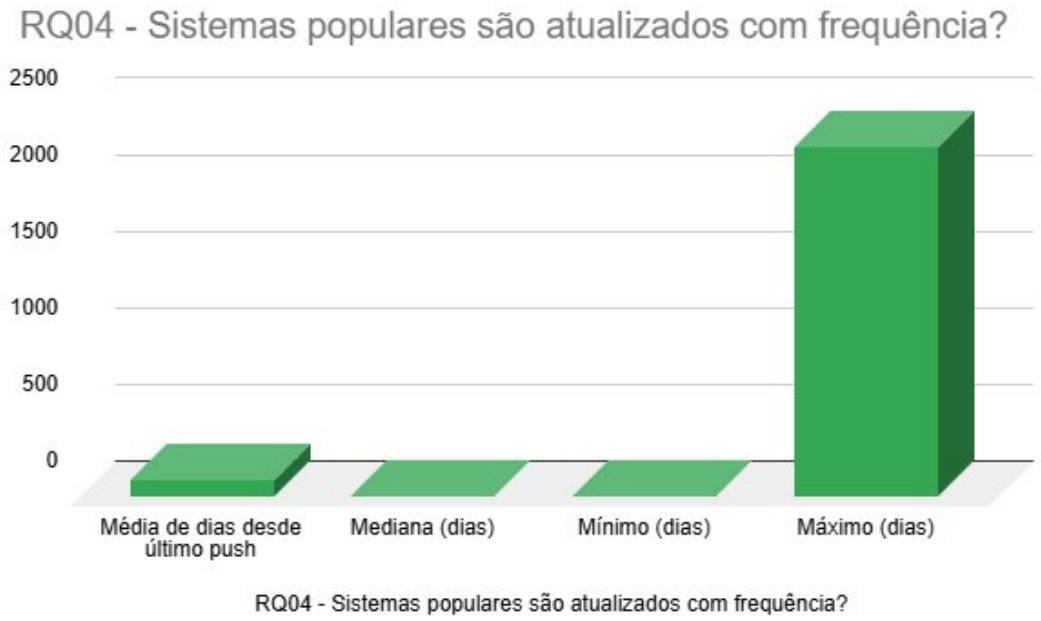

Figura 4 – RQ04: Dias desde último push

RQ05 – Linguagens de Programação

**NOTA – Os percentuais da imagem referem-se apenas às top 10 linguagens; os da tabela são em relação ao total de 1.000 repositórios.*

Pos.	Linguagem	Repositórios	Percentual
1	Python	200	20,00%
2	TypeScript	160	16,00%
3	JavaScript	115	11,50%
4	N/A	95	9,50%
5	Go	77	7,70%
6	Rust	54	5,40%
7	Java	47	4,70%
8	C++	46	4,60%
9	C	25	2,50%
10	Jupyter Notebook	23	2,30%

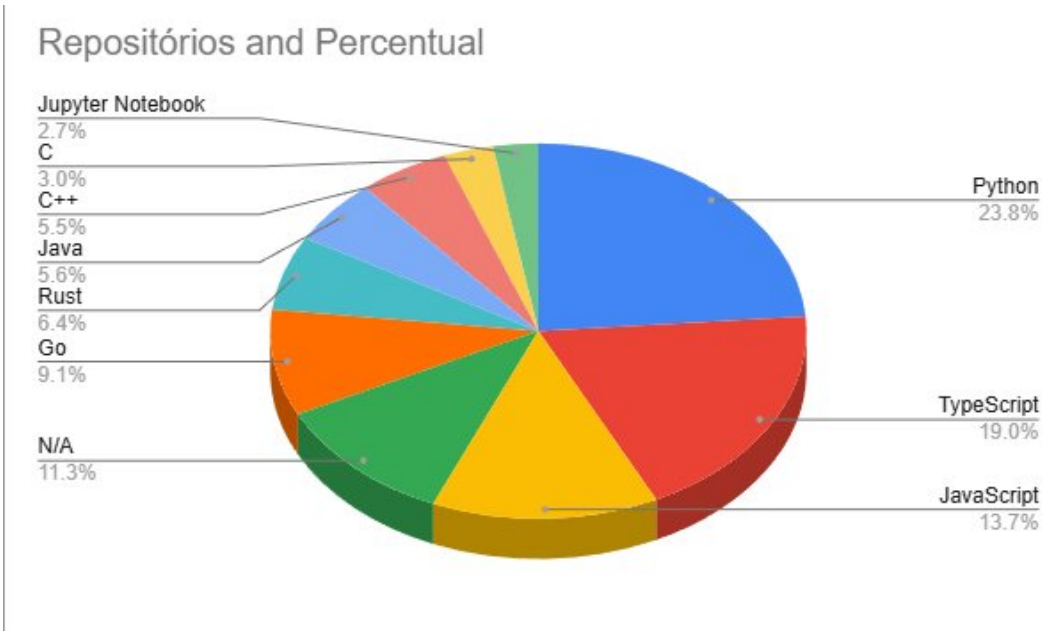

Figura 5 – RQ05: Distribuição de linguagens (repositórios e percentual)

RQ06 – Percentual de Issues Fechadas

Estatística	Valor (%)
Média	77,25%
Mediana	86,8%
Mínimo	0%
Máximo	100%

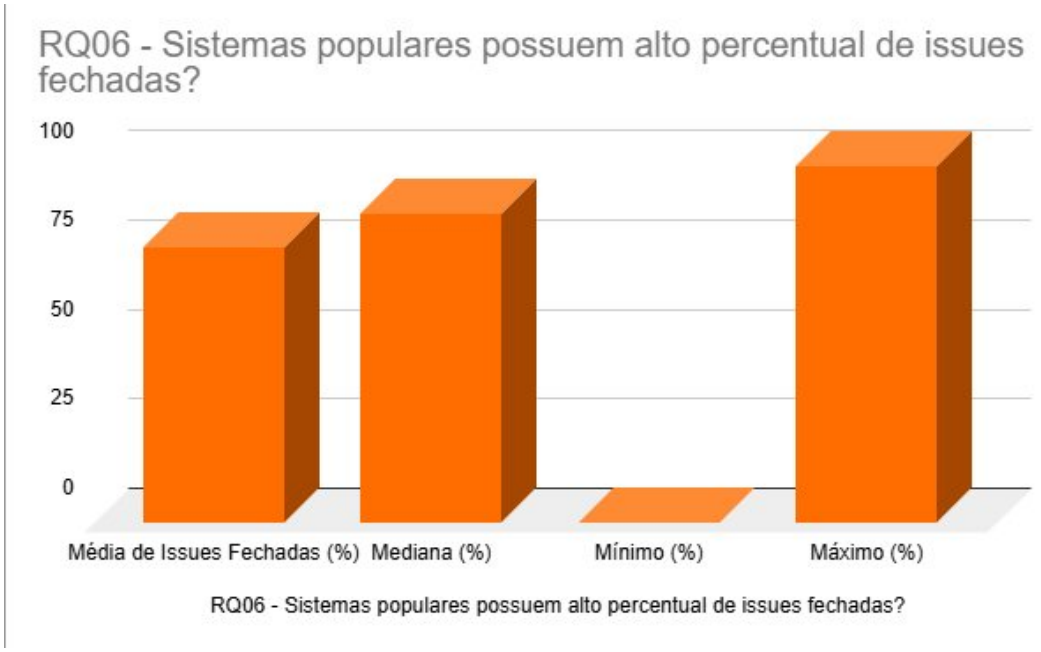

Figura 6 – RQ06: Percentual de issues fechadas

3.3 Análise por Linguagem (RQ07)

Comparação das métricas principais para o Top 5 de linguagens.

Linguagem: Python (Total: 200 repositórios)

Métrica	Média	Mediana
PRs Aceitas	4.059	641
Releases	92	24
Dias desde Push	121	2

Linguagem: TypeScript (Total: 160 repositórios)

Métrica	Média	Mediana
PRs Aceitas	5.023	2.584
Releases	256	158
Dias desde Push	34	0

Linguagem: JavaScript (Total: 115 repositórios)

Métrica	Média	Mediana
PRs Aceitas	2.143	576
Releases	110	40
Dias desde Push	151	4

Linguagem: N/A (Documentação) (Total: 95 repositórios)

Métrica	Média	Mediana
PRs Aceitas	1.331	129
Releases	2	0
Dias desde Push	248	129

Linguagem: Go (Total: 77 repositórios)

Métrica	Média	Mediana
PRs Aceitas	6.053	1.690
Releases	180	132
Dias desde Push	48	0

4. Discussão dos Resultados

4.1 RQ01 – Maturidade dos Sistemas Populares

Hipótese	Sistemas populares tendem a ser maduros, com idade superior a 5 anos (1.825 dias).
Resultado Observado	A mediana de idade foi de 3.062 dias (8,39 anos).
Veredito	■ Confirmada

Análise: Confirmado, grande parte dos sistemas populares tem idade superior a 5 anos, com poucos exemplos abaixo de 5 e menos ainda abaixo de 1 ano de idade. Isso demonstra como os repos populares já têm alta maturidade, conhecimento e tempo na área.

4.2 RQ02 – Contribuição Externa

Hipótese	Sistemas populares recebem alta contribuição externa (mediana > 200 PRs aceitas).
Resultado Observado	A mediana de PRs aceitas foi de 741.
Veredito	■ Confirmada

Análise: Hipótese confirmada. Foi observado que, em geral, há alto índice de PRs, facilmente ultrapassando a estipulação inicial. Existem, no entanto, outliers: há algumas repos sem nenhuma PR e outras com dezenas de milhares (chegando até 94.634!). Isso demonstra que projetos abertos a contribuições tendem a ser mais populares.

4.3 RQ03 – Frequência de Releases

Hipótese	Sistemas populares possuem releases frequentes (mediana > 20 releases).
Resultado Observado	A mediana de releases foi de 41.
Veredito	■ Confirmada (com ressalva)

Análise: Hipótese confirmada, porém com detalhe importante: repos de linguagens de programação (Python, JavaScript, Go...) têm altos níveis de releases, facilmente atingindo ou superando 50. Já repos de documentação (N/A) têm quase nenhuma release. Para aplicações e bibliotecas, o alto versionamento é esperado dado a idade média/mediana elevada dos repos populares.

4.4 RQ04 – Frequência de Atualização

Hipótese	Sistemas populares são atualizados frequentemente, com mediana < 15 dias desde último p
Resultado Observado	A mediana foi de 1 dia desde último push.
Veredito	■ Confirmada

> **Análise:** Confirmada. Grande maioria dos projetos está bem ativa, com dias desde último push variando entre 0 e 5. Há ainda projetos com 100+ dias de inatividade e uma quantidade menor com mais de 300. Seria interessante investigar se os repositórios inativos estão assim por abandono ou por terem atingido sua versão final.

4.5 RQ05 – Linguagens de Programação

Hipótese	Sistemas populares utilizam linguagens mainstream como JavaScript, Python, Java.
Resultado Observado	As 5 linguagens mais frequentes: Python, TypeScript, JavaScript, N/A, Go.
Veredito	■ Confirmada

> **Análise:** Hipótese confirmada. Embora haja dominância do Python, TypeScript e JavaScript, é possível ver como o GitHub é utilizado de outras formas — Go e projetos de documentação (N/A) estão no Top 5. Outras linguagens notáveis como Rust, Java, C++, C e Jupyter Notebook juntas formam 19,5% dos repositórios.

4.6 RQ06 – Gestão de Issues

Hipótese	Sistemas populares possuem alto percentual de issues fechadas (mediana > 70%).
Resultado Observado	A mediana do percentual de issues fechadas foi de 86,8%.
Veredito	■ Confirmada

> **Análise:** Facilmente confirmada. A maioria dos repositórios tem 77% ou mais de issues fechadas. Os outliers são repos sem issues ou com baixo fechamento, mas em quantidade pequena. Projetos populares resolvem a maioria das issues por terem muitos dependentes, alto número de contribuidores e alta maturidade.

4.7 RQ07 – Análise Comparativa por Linguagem

Hipótese	Linguagens diferentes apresentam padrões distintos de contribuição e atualização.
Resultado	Há similaridades e diferenças claras entre grupos de linguagens.
Veredito	■ Confirmada

Padrões observados:

- Python / JavaScript — similares em tempos de push, média/mediana de releases e PRs.
- TypeScript / Go — similares em tempos de push e média/mediana de releases.
- Documentação (N/A) — outlier: tempos longos de push, quase nenhuma release, baixas PRs.

Gráficos Comparativos – Top 5 Linguagens

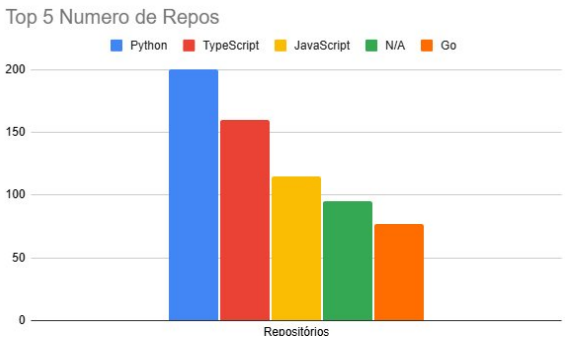

Figura 7 – Top 5 – Número de Repositórios por Linguagem

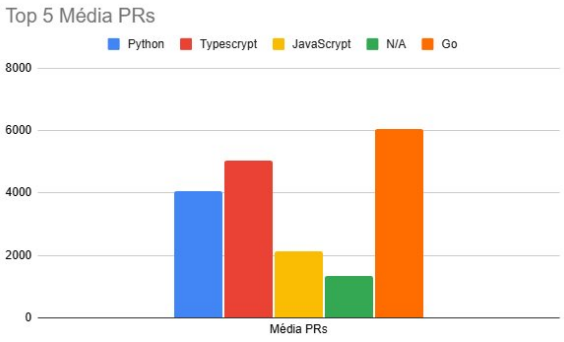

Figura 8 – Top 5 – Média de PRs por Linguagem

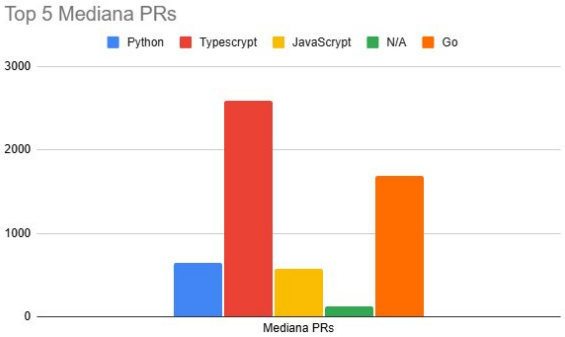

Figura 9 – Top 5 – Mediana de PRs por Linguagem

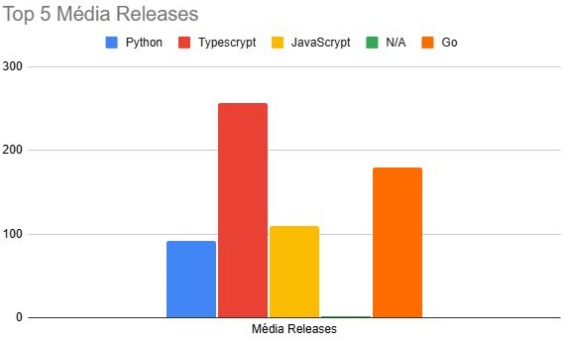

Figura 10 – Top 5 – Média de Releases por Linguagem

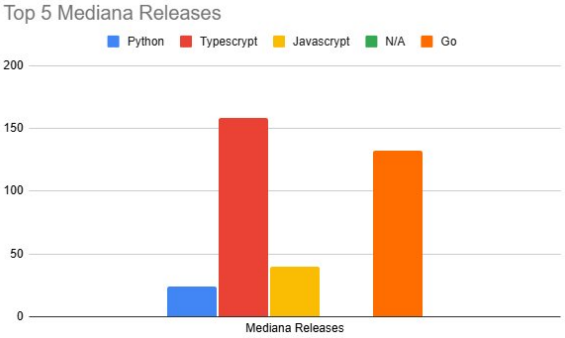

Figura 11 – Top 5 – Mediana de Releases por Linguagem

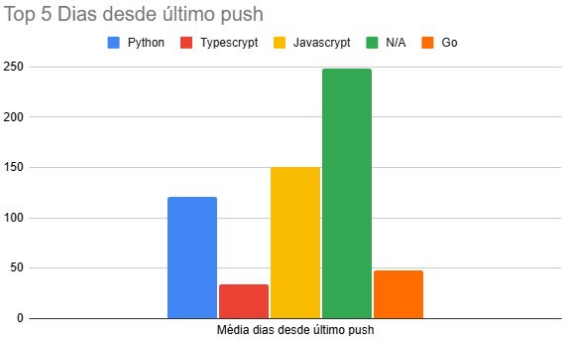

Figura 12 – Top 5 – Média de Dias desde Último Push

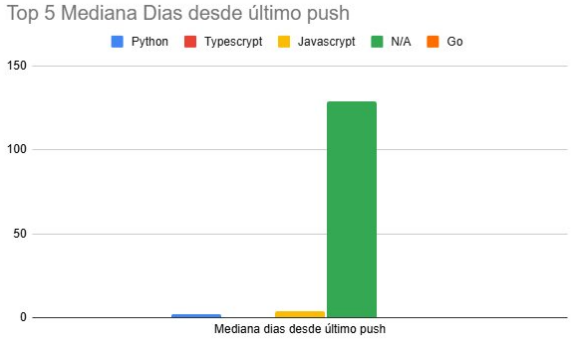

Figura 13 – Top 5 – Mediana de Dias desde Último Push

5. Conclusão

A análise dos 1.000 repositórios mais estrelados do GitHub revelou um conjunto consistente de características que permitem responder às questões de pesquisa e validar a maioria das hipóteses. Em termos de maturidade, a mediana de idade é de 3.062 dias (~8,4 anos), confirmando que projetos populares acumulam reputação ao longo de anos. A mediana de 741 pull requests aceitos por repositório indica elevado nível de colaboração comunitária. A mediana de releases é 41 (acima das 20 esperadas) e a mediana de dias desde o último push é apenas 1 dia, evidenciando bases de código altamente ativas.

Distribuição por Linguagens

Linguagem	% dos Repositórios
Python	20%
TypeScript	16%
JavaScript	11,5%
Go	7,7%
Diversos	44,8%

Repositórios categorizados como documentação (N/A) compõem fração relevante e apresentam comportamento distinto: poucas releases e maior inatividade. A observação dos objetivos de cada repositório deve ser considerada em análises estatísticas.

> **Resumo:** Os resultados devem ser interpretados com cautela pelas limitações conhecidas: seleção por estrelas, análise em snapshot temporal único e categorização por linguagem primária. Ainda assim, os dados fornecem evidências robustas de que repositórios mais estrelados no GitHub tendem a ser projetos consolidados, colaborativos e bem mantidos.

Limitações do Estudo

- Viés de Seleção: Apenas repositórios públicos do GitHub foram analisados.
- Métrica de Popularidade: Número de estrelas pode não refletir uso efetivo ou qualidade técnica.
- Snapshot Temporal: Dados coletados em um único momento não capturam evolução temporal.
- Categorização de Linguagem: Linguagem primária pode não representar linguagens secundárias importantes.
- Contexto de Domínio: Não foi considerado o domínio de aplicação (web, mobile, ML, infraestrutura).

5.1 Tomada de Decisão

- Popularidade: usar número de estrelas como proxy.
- Amostra: analisar top 1.000 repositórios.
- Atualização: medir com pushedAt.
- Contribuições: contar somente PRs MERGED.

- Métrica central: usar mediana.
- Coleta: delay 2s, retry com backoff e salvamento parcial.
- Ferramenta/saída: Python 3.12 + requests, exportar CSV, GUI com tkinter/threading.

5.2 Sugestões Futuras

Dados Adicionais

- Contribuidores: total de contribuidores, novos por mês e métricas de retenção.
- Atividade de commits: commits por período, churn e tamanho médio de PR.
- Qualidade de código: cobertura de testes, taxa de sucesso de CI, métricas de lint e complexidade.

Novos Indicadores

- Taxa de aceitação de PRs: PRs MERGED / PRs abertos.
- Cadência de releases: tempo médio entre releases, releases por ano, aderência ao versionamento semântico.
- Coleta longitudinal: snapshots periódicos para análises de tendência.
- Exportação rica: além de CSV, exportar JSON com metadados da query e timestamp.

Coletas

- Aumentar o número de repositórios coletados, gerando amostra mais precisa e significativa.
- Utilizar métrica parametrizável ao invés de apenas por número de estrelas.